

Вейвлет (маленькая волна) – это функция, которая выглядит как короткий колебательный сигнал (в отличие от синусоиды, которая бесконечна). Вейвлеты позволяют анализировать сигнал локально (в определенные моменты времени).

Ключевые свойства вейвлетов – Ограничены во времени и частоте. Форма сохраняется при масштабировании.

Примеры вейвлетов: Хаара – дискретный вейвлет, Добеши – ортогональные вейвлеты.

Фурье показывает только частоты, но не их временное положение. Вейвлеты дают частотно-временную карту сигнала.

Дискретное вейвлет-преобразование (DWT)

Это алгоритм, который разбивает сигнал на аппроксимирующую (A) и детализирующую (D) части с помощью фильтров. Аппроксимация (A) – грубая, сглаженная версия сигнала (основная структура). Детали (D) – высокочастотные компоненты (шум, резкие изменения).

Вейвлет-преобразование демонстрирует сравнения сигнала с заданными базовыми вейвлетами в каждый момент времени существования сигнала. Хорошо приспособлено для изучения структуры неоднородных сигналов.

Применяется в задачах распознавания образов, шумоподавление, сжатие данных, анализ и обработка сигналов и изображений (радужная оболочка глаз, рентгенограмма, спутниковые снимки).

Работает как многоступенчатый фильтр: на каждом уровне сигнал пропускают через два фильтра (для плавных и резких изменений), затем оставляют только каждый второй отсчёт – это уменьшает данные вдвое. При восстановлении нули вставляют между оставшимися отсчётами и пропускают через обратные фильтры, чтобы вернуть исходный размер без потери информации. Обрезка краёв нужна, чтобы убрать искажения от свёртки.

D2 означает, что у вейвлета 2 vanishing moments (два нулевых момента), что влияет на его способность подавлять шум.

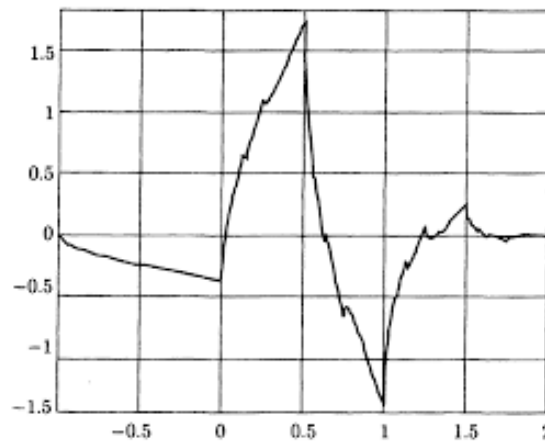
Нулевые моменты – это свойство вейвлета "игнорировать" полиномиальные составляющие сигнала. Чем больше нулевых моментов, тем лучше вейвлет подавляет гладкие части сигнала и выделяет резкие изменения.

Уровень (L=4) – уровень\глубина разложения сигнала. Каждый уровень делит сигнал на две части: A – аппроксимация (низкие частоты) и D – детали (высокие частоты).

Обнуление коэффициентов – Пример: Музыка. Аппроксимация (A4): Основная мелодия. Детали (D1-D4): Шум и высокочастотные артефакты. Обнуляем D3: Шум исчезает, звук становится немного "мягче".

Чем глубже уровень, тем более "грубую" информацию (низкие частоты) он содержит. Детали (D) на высоких уровнях обычно соответствуют шуму.

Материнский вейвлет – Это базовая функция, из которой генерируются все дочерние вейвлеты путем масштабирования и сдвига.



Фильтры – это наборы коэффициентов, которые используются для разложения (декомпозиции) и восстановления (реконструкции) сигнала. Фильтры позволяют разделить сигнал на частотные компоненты и затем восстановить его. Без них невозможно выполнить DWT.

Фильтры декомпозиции (`dec_lo`, `dec_hi`) – это коэффициенты, которые применяются к сигналу для разделения на низкочастотную (аппроксимация) и высокочастотную (детали) компоненты.

Фильтры реконструкции (`rec_lo`, `rec_hi`) – обратные фильтры, которые нужны для восстановления сигнала из коэффициентов.

Когда выполняем свертку (`np.convolve`), фильтр "скользит" по сигналу, и на краях сигнала происходит следующее: Фильтр "портит" данные на половину своей длины с каждого края. $4 \setminus 2 = 2$

Для Db2 коэффициенты были найдены Ингрид Добеши в 1988 году с помощью решения системы уравнений. Используем из библиотеки, эти самые.

Без коррекции – восстановленный сигнал почти совпадает с зашумленным (как и должно быть, разложили и собрали его обратно).

После коррекции – шум уменьшается, сигнал становится ближе к исходному, но могут теряться некоторые детали из-за обнуления коэффициентов.

Т.е. проводим селективное шумоподавление, и результат можно увидеть на графиках – восстановленный сигнал стал менее зашумленным.

Db2 использует 4 коэффициента в каждом фильтре. Это означает, что при свертке сигнала с фильтром учитываются только 4 соседних отсчета. Менее точное частотное разделение по сравнению с длинными фильтрами (Db8).

Непрерывное вейвлет-преобразование (CWT) – это метод анализа сигналов, который использует непрерывное масштабирование и сдвиг материнского вейвлета для изучения сигнала в разных частотных диапазонах и временных точках. Нужен для анализа нестационарных сигналов (где частоты меняются со временем). CWT работает с произвольными масштабами и сдвигами, а DWT – только с дискретными (степени двойки). Пример: Анализ электрокардиограммы (ЭКГ).

Алгоритм:

1. Генерация сигнала

```
n_samples = 1024 # количество отсчетов
t = np.linspace(0, 1, n_samples) # временная ось от 0 до 1 сек
f = np.sin(2 * np.pi * t) + np.cos(4 * 2 * np.pi * t) # сигнал = синус + косинус
```

2. Добавление шума

```
noise = np.random.uniform(-1, 1, n_samples) # шум от -1 до 1
f_noisy = f + noise # зашумленный сигнал
```

3. Прямое вейвлет-преобразование (DWT)

```
def dwt(signal, lo_filter, hi_filter):
    pad = len(lo_filter) # размер дополнения
    padded = np.pad(signal, (pad // 2, pad - pad // 2), mode='symmetric') # симметричное дополнение
    lo = np.convolve(padded, lo_filter, mode='valid')[:,2] # свертка + децимация (прореживание)
    hi = np.convolve(padded, hi_filter, mode='valid')[:,2]
    return lo, hi
```

4. Многоуровневое разложение (4 уровня)

```
coeffs = []
current = f_noisy.copy()
for _ in range(4): # 4 уровня
    a, d = dwt(current, lo_d, hi_d) # разложение
    coeffs.append((a, d)) # сохранение коэффициентов
    current = a # следующий уровень работает с аппроксимацией
```

5. Обратное преобразование (IDWT)

```
def idwt(a, d, lo_filter, hi_filter):
    a_up = np.zeros(2 * len(a)) # интерполяция нулями
    a_up[::2] = a # вставка коэффициентов
    d_up = np.zeros(2 * len(d))
    d_up[::2] = d
    # Свертка с обратными фильтрами
    a_conv = np.convolve(a_up, lo_filter, mode='full')
    d_conv = np.convolve(d_up, hi_filter, mode='full')
    # Суммирование и обрезка
    reconstructed = a_conv[:min(len(a_conv), len(d_conv))] + d_conv[:min(len(a_conv), len(d_conv))]
    trim = len(lo_filter) // 2
    return reconstructed[trim:-trim] # обрезка краевых эффектов
```

Интерполяция нулями – увеличиваем длину массива в 2 раза.

Свертка + суммирование – восстанавливаем сигнал.

6. Модификация коэффициентов

```
modified_coeffs = [list(c) for c in coeffs] # копируем коэффициенты
modified_coeffs[3][1] *= 0.5 # L4 детали уменьшаем в 2 раза
modified_coeffs[2][1] = np.zeros_like(modified_coeffs[2][1]) # L3 детали обнуляем
```

L4 (4-й уровень деталей) – уменьшаем вклад высоких частот.

L3 (3-й уровень деталей) – полностью убираем (этот шум сильнее всего искажает сигнал).

Импорт необходимых библиотек

```
import numpy as np # Для работы с массивами и математическими операциями
import matplotlib.pyplot as plt # Для визуализации графиков
import ruwt # Библиотека для вейвлет-преобразований (используется только для
получения фильтров)
```

Генерация тестового сигнала

```
n_samples = 1024 # Количество отсчетов сигнала
t = np.linspace(0, 1, n_samples) # Временная шкала от 0 до 1 с n_samples точками
f = np.sin(2 * np.pi * t) + np.cos(4 * 2 * np.pi * t) # Исходный сигнал - сумма
синуса и косинуса
```

```

# Добавление шума к сигналу
np.random.seed(42) # Фиксируем seed для воспроизводимости результатов
noise = np.random.uniform(-1, 1, n_samples) # Генерируем равномерный шум в
диапазоне [-1, 1]
f_noisy = f + noise # Добавляем шум к исходному сигналу

# Визуализация исходного и зашумленного сигналов
plt.figure(figsize=(10, 6)) # Создаем фигуру заданного размера
plt.plot(t, f, label='Исходный сигнал') # График исходного сигнала
plt.plot(t, f_noisy, label='Зашумленный', alpha=0.7) # График зашумленного
сигнала с прозрачностью
plt.grid(True) # Включаем сетку
plt.legend() # Добавляем легенду
plt.title('Сравнение сигналов') # Заголовок графика
plt.show() # Отображаем график

# Получаем коэффициенты фильтров Добеши 2-го порядка из библиотеки pywt (только
для справки)
print(pywt.Wavelet('db2').dec_lo) # Выводим низкочастотный фильтр декомпозиции
print(pywt.Wavelet('db2').dec_hi) # Выводим высокочастотный фильтр декомпозиции

# Сохраняем коэффициенты фильтров в массивы numpy
lo_d = np.array(pywt.Wavelet('db2').dec_lo) # Низкочастотный фильтр декомпозиции
hi_d = np.array(pywt.Wavelet('db2').dec_hi) # Высокочастотный фильтр
декомпозиции
lo_r = lo_d[::-1] # Обратный низкочастотный фильтр для реконструкции
hi_r = hi_d[::-1] # Обратный высокочастотный фильтр для реконструкции

# Функция прямого вейвлет-преобразования (DWT)
def dwt(signal, lo_filter, hi_filter):
    pad = len(lo_filter) # Определяем необходимый размер дополнения
    padded = np.pad(signal, (pad // 2, pad - pad // 2), mode='symmetric') #
Дополняем сигнал симметрично
    lo = np.convolve(padded, lo_filter, mode='valid')[:,2] # Применяем
низкочастотный фильтр и децимацию
    hi = np.convolve(padded, hi_filter, mode='valid')[:,2] # Применяем
высокочастотный фильтр и децимацию
    return lo, hi # Возвращаем аппроксимирующие и детализирующие коэффициенты

# Функция многоуровневого вейвлет-разложения
def multilevel_dwt(signal, levels):
    coeffs = [] # Список для хранения коэффициентов всех уровней
    current = signal.copy() # Копируем сигнал, чтобы не изменять оригинал
    for _ in range(levels): # Для каждого уровня разложения
        a, d = dwt(current, lo_d, hi_d) # Выполняем DWT
        coeffs.append((a, d)) # Сохраняем коэффициенты
        current = a # Для следующего уровня используем аппроксимацию
    return coeffs # Возвращаем все коэффициенты

# Выполняем 4-уровневое разложение зашумленного сигнала

```

```

coeffs = multilevel_dwt(f_noisy, 4)

# Визуализация коэффициентов разложения
plt.figure(figsize=(12, 10)) # Создаем большую фигуру
titles = ['Уровень 1', 'Уровень 2', 'Уровень 3', 'Уровень 4'] # Заголовки для графиков
for i in range(4): # Для каждого уровня
    plt.subplot(4, 2, 2 * i + 1) # Создаем подграфик для аппроксимации
    plt.plot(coeffs[i][0]) # График аппроксимирующих коэффициентов
    plt.title(f'Аппроксимация L{i + 1}') # Заголовок
    plt.grid(True) # Включаем сетку

    plt.subplot(4, 2, 2 * i + 2) # Создаем подграфик для деталей
    plt.plot(coeffs[i][1]) # График детализирующих коэффициентов
    plt.title(f'Детали L{i + 1}') # Заголовок
    plt.grid(True) # Включаем сетку
plt.tight_layout() # Автоматическая настройка расположения подграфиков
plt.show() # Отображаем графики

# Функция обратного вейвлет-преобразования (IDWT)
def idwt(a, d, lo_filter, hi_filter):
    # Восстановление нулей (интерполяция)
    a_up = np.zeros(2 * len(a)) # Создаем массив с нулями в 2 раза больше
    a_up[::2] = a # Вставляем коэффициенты через один отсчет
    d_up = np.zeros(2 * len(d)) # Аналогично для деталей
    d_up[::2] = d # Вставляем коэффициенты

    # Свертка с фильтрами реконструкции
    a_conv = np.convolve(a_up, lo_filter, mode='full') # Низкочастотная компонента
    d_conv = np.convolve(d_up, hi_filter, mode='full') # Высокочастотная компонента

    # Суммируем компоненты и обрезаем лишнее
    min_length = min(len(a_conv), len(d_conv)) # Определяем минимальную длину
    reconstructed = a_conv[:min_length] + d_conv[:min_length] # Суммируем

    # Обрезаем краевые эффекты
    trim = len(lo_filter) // 2 # Сколько отсчетов обрезать
    return reconstructed[trim:-trim] # Возвращаем обрезанный сигнал

# Функция полной реконструкции сигнала из коэффициентов
def reconstruct(coeffs):
    current_a = coeffs[-1][0] # Начинаем с аппроксимации последнего уровня
    for i in reversed(range(len(coeffs) - 1)): # Идем от старших уровней к младшим
        current_a = idwt(current_a, coeffs[i][1], lo_r, hi_r) # Восстанавливаем уровень
    return idwt(current_a, coeffs[0][1], lo_r, hi_r)[:len(f_noisy)] # Финальное восстановление и обрезка

```

```
# Восстанавливаем сигнал без изменений коэффициентов
f_restored = reconstruct(coeffs)

# Визуализация зашумленного и восстановленного сигналов
plt.figure(figsize=(12, 5)) # Создаем фигуру
plt.subplot(211) # Верхний график
plt.plot(t, f_noisy) # Зашумленный сигнал
plt.title('Зашумленный сигнал') # Заголовок
plt.grid(True) # Сетка

plt.subplot(212) # Нижний график
plt.plot(t, f_restored) # Восстановленный сигнал
plt.title('Восстановленный без коррекции') # Заголовок
plt.grid(True) # Сетка
plt.tight_layout() # Автонастройка
plt.show() # Отображение

# Модификация коэффициентов для шумоподавления
modified_coefs = [list(c) for c in coeffs] # Копируем коэффициенты
modified_coefs[3][1] = modified_coefs[3][1] * 0.5 # Уменьшаем детали 4-го
уровня вдвое
modified_coefs[2][1] = np.zeros_like(modified_coefs[2][1]) # Обнуляем детали
3-го уровня

# Восстановление сигнала с модифицированными коэффициентами
f_denoised = reconstruct(modified_coefs)

# Финальное сравнение исходного и очищенного сигналов
plt.figure(figsize=(12, 5)) # Создаем фигуру
plt.subplot(211) # Верхний график
plt.plot(t, f, label='Исходный') # Исходный сигнал
plt.title('Исходный сигнал') # Заголовок
plt.grid(True) # Сетка

plt.subplot(212) # Нижний график
plt.plot(t, f_denoised, label='Очищенный') # Очищенный сигнал
plt.title('Результат обработки') # Заголовок
plt.grid(True) # Сетка
plt.tight_layout() # Автонастройка
plt.show() # Отображение
```